



Telegram Bot Service & Ephemeral Video Engine — Codebase

[PRODUCTION SOURCE CODE](#)

✓ 100% SANITIZED (NO KEYS / NO TOKENS)

• Fayl: backend/bot_service.py • Jami qatorlar: 1,989 ta

```
1  """Course Academy Telegram Bot.
2
3  Aiogram ishlatmasdan Telegram Bot API + httpx asosida yozilgan: FastAPI bilan bitta
4  event loopda ishlaydi, ammo /start, katalog, checkout FSM, chek tasdiqlash, join
5  request himoyasi va admin buyruqlari to'liq ajratilgan handlerlarga ega.
6  """
7  from __future__ import annotations
8
9  import asyncio
10 import datetime as _dt
11 import html
12 import json
13 import logging
14 import time
15 import uuid
16 from typing import Any, Dict, Optional, Tuple, List
17
18 import httpx
19
20 from app.api.ai import call_openrouter_api
21 from app.core.config import settings
22 from app.core.r2 import r2_client
23 from app.services.pricing import course_pricing
24 from app.services.purchases import (
25     approve_purchase,
26     is_join_request_authorized,
27     reject_purchase,
28     revoke_join_request_link,
29 )
30 from app.storage import get_store
31
32 logger = logging.getLogger(__name__)
33
34 API_URL = f"https://api.telegram.org/bot[REDACTED_TOKEN]""[REDACTED_BOT_TOKEN]"
35
36 # FSM faqat chek yuborishning qisqa oralig'i uchun kerak. Jarayon server qayta
37 # ishga tushsa toza holatga qaytadi; tasdiqlangan cheklar esa doim bazada qoladi.
38 _checkout_sessions: Dict[int, str] = {}
39
40 # Guruh AI yordamchisi uchun chat bo'yicha tezlik cheklovi (spam/kredit himoyasi)
41 _GROUP_AI_STATE: Dict[int, Dict[str, Any]] = {}
42 BOT_ID: Optional[int] = None
43
44 # Guruhda AI savol-javobni ishga tushiradigan sotuvga aloqador so'zlar
45 _GROUP_SALES_KEYWORDS = (
46     "kurs", "narx", "narxi", "chegirma", "to'lov", "tolov", "o'qimoq", "o'qimoqchi",
47     "o'rgan", "sertifikat", "platforma", "kreativ", "dizayn", "smm", "dasturlash",
48     "sun'iy intellekt", "prompt", "nechchi dars", "qancha turadi",
49 )
50
51 # Adminlar video yuklash holati: admin_id -> video_meta_dict
52 _pending_admin_videos: Dict[int, Dict[str, Any]] = {}
53
54 # Studentlar uchun dars ko'rish rate limiting: (user_id, lesson_id) -> last_click_timestamp
55 _user_lesson_rate_limits: Dict[Tuple[int, str], float] = {}
56
57
58 def _uzs(amount: Any) -> str:
59     try:
60         return f"{int(amount or 0):,}".replace(",", " ") + " so'm"
61     except (ValueError, TypeError):
62         return "0 so'm"
63
64
65 def _escape(value: Any) -> str:
66     return html.escape(str(value or ""))
67
68
69 async def _telegram_call(
70     client: httpx.AsyncClient, method: str, payload: Dict[str, Any]
71 ) -> Dict[str, Any]:
72     """Telegram so'rovini bajaradi va xatoni logga yozadi."""
73     try:
74         response = await client.post(f"{API_URL}/{method}", json=payload)
75         data = response.json()
```

```

76         if response.status_code != 200 or not data.get("ok"):
77             logger.warning("Telegram %s xatosi: %s", method, response.text[:300])
78             return data if isinstance(data, dict) else {}
79     except (httpx.HTTPError, ValueError) as exc:
80         logger.error("Telegram %s so'rovida xato: %s", method, exc)
81         return {}
82
83
84     async def send_tg_message(
85         client: httpx.AsyncClient,
86         chat_id: int,
87         text: str,
88         reply_markup: Optional[dict] = None,
89         protect_content: bool = True,
90     ) -> Dict[str, Any]:
91         payload: Dict[str, Any] = {
92             "chat_id": chat_id,
93             "text": text,
94             "parse_mode": "HTML",
95             "protect_content": protect_content,
96             "disable_web_page_preview": True,
97         }
98         if reply_markup:
99             payload["reply_markup"] = reply_markup
100         return await _telegram_call(client, "sendMessage", payload)
101
102
103     async def send_tg_photo(
104         client: httpx.AsyncClient,
105         chat_id: int,
106         photo: str,
107         caption: str,
108         reply_markup: Optional[dict] = None,
109     ) -> Dict[str, Any]:
110         payload: Dict[str, Any] = {
111             "chat_id": chat_id,
112             "photo": photo,
113             "caption": caption[:1024],
114             "parse_mode": "HTML",
115             "protect_content": True,
116         }
117         if reply_markup:
118             payload["reply_markup"] = reply_markup
119         return await _telegram_call(client, "sendPhoto", payload)
120
121
122     def is_admin(user_id: int | str) -> bool:
123         """str va int 64-bit xavfsiz admin tekshiruvi"""
124         if not user_id:
125             return False
126         try:
127             uid_int = int(str(user_id).strip())
128             if uid_int in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
129                 return True
130         except (ValueError, TypeError):
131             pass
132         uid_str = str(user_id).strip()
133         return uid_str in [str(a) for a in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]]
134
135
136     async def send_tg_video(
137         client: httpx.AsyncClient,
138         chat_id: int | str,
139         video: str,
140         caption: Optional[str] = None,
141         reply_markup: Optional[dict] = None,
142         protect_content: bool = True,
143         supports_streaming: bool = True,
144         duration: Optional[int] = None,
145         width: Optional[int] = None,
146         height: Optional[int] = None,
147     ) -> Dict[str, Any]:
148         payload: Dict[str, Any] = {
149             "chat_id": chat_id,
150             "video": video,
151             "protect_content": protect_content,
152             "supports_streaming": supports_streaming,
153         }
154         if caption:
155             payload["caption"] = caption[:1024]
156             payload["parse_mode"] = "HTML"

```

```

157     if reply_markup:
158         payload["reply_markup"] = reply_markup
159     if duration:
160         payload["duration"] = int(duration)
161     if width:
162         payload["width"] = int(width)
163     if height:
164         payload["height"] = int(height)
165     return await _telegram_call(client, "sendVideo", payload)
166
167
168 async def send_ephemeral_video(
169     client: httpx.AsyncClient,
170     chat_id: int | str,
171     video_file_id: str,
172     caption: str,
173     callback_user_id: int,
174     callback_query_id: str,
175     replace_callback_query_message: bool = False,
176     duration: Optional[int] = None,
177     width: Optional[int] = None,
178     height: Optional[int] = None,
179     protect_content: bool = True,
180 ) -> Dict[str, Any]:
181     """
182     Telegram Ephemeral Protected Video:
183     Telegram Bot API sendVideo chaqiruvi ephemeral_message_parameters bilan
184     faqat tugmani bosgan studentga va faqat o'sha clientga yuboriladi.
185     protect_content=True qat'iy qo'llaniladi.
186     """
187     payload: Dict[str, Any] = {
188         "chat_id": chat_id,
189         "video": video_file_id,
190         "caption": caption[:1024],
191         "supports_streaming": True,
192         "protect_content": protect_content,
193         "ephemeral_message_parameters": {
194             "receiver_user_id": int(callback_user_id),
195             "callback_query_id": str(callback_query_id),
196             "replace_callback_query_message": bool(replace_callback_query_message),
197         }
198     }
199     if duration:
200         payload["duration"] = int(duration)
201     if width:
202         payload["width"] = int(width)
203     if height:
204         payload["height"] = int(height)
205
206     return await _telegram_call(client, "sendVideo", payload)
207
208
209 async def send_tg_chat_action(client: httpx.AsyncClient, chat_id: int, action: str = "typing") -> None:
210     await _telegram_call(client, "sendChatAction", {"chat_id": chat_id, "action": action})
211
212
213 def _resolve_media_url(url: str) -> str:
214     """R2 media havolalarini Telegram yubora oladigan to'g'ridan-to'g'ri URL ga aylantiradi.
215
216     /api/media/{key} havolalari 307 redirect qaytaradi, Telegram sendPhoto esa
217     redirectlarni kuzatmaydi – shuning uchun kalit bo'yicha presigned URL ochamiz.
218     """
219     resolved = r2_client.resolve_stream_url(url, expires_in=86400)
220     return str(resolved or "").strip()
221
222
223 async def _answer_callback(client: httpx.AsyncClient, query_id: str, text: str = "") -> None:
224     payload: Dict[str, Any] = {"callback_query_id": query_id}
225     if text:
226         payload["text"] = text[:190]
227     await _telegram_call(client, "answerCallbackQuery", payload)
228
229
230 async def _ensure_user(tg_user: Dict[str, Any]) -> Optional[Dict[str, Any]]:
231     """Botdan foydalangan odamni users jadvalida yaratadi/yangi ma'lumotini yangilaydi."""
232     telegram_id = tg_user.get("id")
233     if not telegram_id:
234         return None
235
236     store = get_store()
237     name = " ".join(

```

```

238         part for part in (tg_user.get("first_name"), tg_user.get("last_name")) if part
239     ).strip() or "Talaba"
240     username = tg_user.get("username")
241     user = await store.get_user_by_tg(int(telegram_id))
242     if user:
243         updates = {
244             key: value
245             for key, value in {"name": name, "username": username}.items()
246             if value is not None and user.get(key) != value
247         }
248         if updates:
249             await store.update_user(user["id"], updates)
250             user.update(updates)
251         return user
252
253     role = "superadmin" if int(telegram_id) in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"] else "student"
254     try:
255         return await store.create_user(
256             {"telegram_id": int(telegram_id), "name": name, "username": username, "role": role}
257         )
258     except Exception as exc:
259         # Ikki update parallel kelgan bo'lsa, ikkinchisi mavjud userni qayta oladi.
260         logger.warning("Telegram user yaratilmadi: %s", exc)
261         return await store.get_user_by_tg(int(telegram_id))
262
263
264 def _course_keyboard(course: Dict[str, Any], index: int, total: int) -> dict:
265     rows = [
266         [
267             {"text": "🔍 Oldingi", "callback_data": f"course:{(index - 1) % total}"},
268             {"text": f"{index + 1}/{total}", "callback_data": "noop"},
269             {"text": "Keyingi ➡", "callback_data": f"course:{(index + 1) % total}"},
270         ],
271         [{"text": "📄 To'lov qilish", "callback_data": f"pay:{course['id']}"}],
272         [{"text": "📱 Mini Appda batafsil", "web_app": {"url": f"{settings.WEBAPP_URL}#course_{course['id']}"}}],
273     ]
274     return {"inline_keyboard": rows}
275
276
277 def _course_caption(course: Dict[str, Any], pricing: Optional[Dict[str, Any]] = None) -> str:
278     title = _escape(course.get("title") or "Kurs")
279     category = _escape(course.get("category") or "Premium ta'lim")
280     instructor = _escape(course.get("instructor_name") or "Kreativ AI ustozlari")
281
282     instructor_title = _escape(course.get("instructor_title") or "Ekspert")
283     # Faol "birinchi N kishi" chegirmasi bo'lsa – chegirmali yakuniy narx ko'rsatiladi
284     discount_active = bool((pricing or {}).get("discount_active"))
285     final_price = (pricing or {}).get("final_price")
286     shown_price = final_price if discount_active and final_price is not None else course.get("price")
287     shown_old = course.get("price") if discount_active and final_price is not None else course.get("old_price")
288     old_price_line = f"\n👉 <s>{_uzs(shown_old)}</s>" if shown_old and shown_old > (shown_price or 0) else ""
289     discount = course.get("discount_percent")
290     if discount_active and discount:
291         spots = (pricing or {}).get("discount_spots_left")
292         discount_line = f"\n🔥 <b>--(int(discount))% chegirma</b> – birinchi {int(course.get('discount_limit') or 0)} kishi uchun"
293         if spots is not None:
294             discount_line += f", {int(spots)} ta joy qoldi"
295     else:
296         discount_line = f" 🧡 --(discount)%" if discount and not course.get("discount_limit") else ""
297     description = _escape(course.get("short_description") or course.get("description") or "")
298     if len(description) > 290:
299         description = description[:287].rstrip() + "..."
300     return (
301         f"📌 <b>{title}</b>\n"
302         f"📂 {category}\n"
303         f"📅 {int(course.get('lesson_count') or 0)} ta dars · ⌚ {_escape(course.get('duration') or 'Davomiyligi ko'rsatiladi')}\n"
304         f"💰 <b>{_uzs(shown_price)}</b>{old_price_line}{discount_line}\n"
305         f"👤 <b>{instructor}</b> – {instructor_title}\n"
306         f"⭐ {course.get('rating') or 5.0}/5\n"
307         f"{description}"
308     )
309
310
311 async def show_course_card(client: httpx.AsyncClient, chat_id: int, index: int) -> None:
312     store = get_store()
313     courses = await store.list_courses(published_only=True)
314     if not courses:
315         await send_tg_message(client, chat_id, "📅 Hozircha kurslar tayyorlanmoqda. Tez orada qayta ko'ring!")
316         return
317
318     index %= len(courses)

```

```

319     course = courses[index]
320     try:
321         pricing = await course_pricing(store, course)
322     except Exception:
323         pricing = {"discount_active": False, "discount_spots_left": None, "final_price": course.get("price")}
324     keyboard = _course_keyboard(course, index, len(courses))
325     caption = _course_caption(course, pricing)
326     cover_url = _resolve_media_url(course.get("cover_url") or "")
327     if cover_url.startswith(("https://", "http://")):
328         result = await send_tg_photo(client, chat_id, cover_url, caption, keyboard)
329         if result.get("ok"):
330             return
331     await send_tg_message(client, chat_id, caption, keyboard)
332
333
334 async def _get_active_card_info() -> Dict[str, str]:
335     store = get_store()
336     card_num = settings.CARD_NUMBER
337     card_holder = settings.CARD_HOLDER
338     bank_name = settings.CARD_BANK
339     try:
340         saved_pay = await store.get_setting("payment_settings")
341         if saved_pay:
342             data = json.loads(saved_pay)
343             if data.get("card_number"):
344                 card_num = data["card_number"]
345             if data.get("card_holder"):
346                 card_holder = data["card_holder"]
347             if data.get("bank_name"):
348                 bank_name = data["bank_name"]
349     except Exception:
350         pass
351     return {"card_number": card_num, "card_holder": card_holder, "bank_name": bank_name}
352
353
354 async def _send_payment_info(client: httpx.AsyncClient, chat_id: int) -> None:
355     card = await _get_active_card_info()
356     text = (
357         "📄 <b>Kreativ AI – To'lov rekvizitlari</b>\n\n"
358         f"🏦 Bank: <b>{_escape(card['bank_name'])}</b>\n"
359         f"📄 Karta raqami (nusxalash uchun bosing):\n"
360         f"<code>{_escape(card['card_number'])}</code>\n"
361         f"👤 Qabul qiluvchi: <b>{_escape(card['card_holder'])}</b>\n\n"
362         "💡 <b>To'lov tartibi:</b>\n"
363         "1. Karta raqamidan nusxa oling va ilovangizdan pul o'tkazing.\n"
364         "2. To'lov chekini skrinshot qilib, shu chatga rasm ko'rinishida yuboring.\n"
365         "3. Admin tasdiqlashi bilan darslar va yopiq kanal avtomatik ochiladi."
366     )
367     rows = [
368         [{"text": "📄 Kreativ AI – Mini App", "web_app": {"url": settings.WEBAPP_URL}},
369         [{"text": "📄 Kurslar Katalogi", "callback_data": "course:0"}],
370     ]
371     await send_tg_message(
372         client,
373         chat_id,
374         text,
375         {"inline_keyboard": rows},
376     )
377
378
379 async def _start_checkout(client: httpx.AsyncClient, chat_id: int, user: Dict[str, Any], course_id: str) -> None:
380     store = get_store()
381     course = await store.get_course(course_id)
382     if not course or not course.get("published", True):
383         await send_tg_message(client, chat_id, "🚫 Bu kurs hozir mavjud emas.")
384         return
385     card = await _get_active_card_info()
386     # Chegirma faol bo'lsa bot ham mini-app bilan bir xil yakuniy narxni ko'rsatadi
387     try:
388         pricing = await course_pricing(store, course)
389     except Exception:
390         pricing = {"discount_active": False, "discount_spots_left": None, "final_price": course.get("price")}
391     amount = pricing["final_price"]
392     price_block = f"💰 Summa: <b>{_uzs(amount)}</b>"
393     if pricing["discount_active"] and course.get("discount_percent"):
394         price_block = (
395             f"💰 Summa: <s>{_uzs(course.get('price'))}</s> → <b>{_uzs(amount)}</b>\n"
396             f"🔥 <b>{-int(course['discount_percent'])}% chegirma</b> – birinchi {int(course.get('discount_limit') or 0)} kishi uchun"
397             + (f", {int(pricing['discount_spots_left'])} ta joy qoldi" if pricing.get("discount_spots_left") is not None else "")
398         )

```

```

399 # Hamyon balansi: bot oqimida hamyondan yechish yo'q (bu faqat Mini App checkoutda),
400 # shuning uchun balans bor foydalanuvchiga aniq yo'l-yo'rig' ko'rsatamiz.
401 wallet_line = ""
402 try:
403     from app.services import wallet as wallet_service
404     wallet = await wallet_service.get_wallet(store, str(user.get("id") or ""))
405     if wallet.get("balance", 0) > 0:
406         wallet_line = (
407             f"\n 📁 <b>Hamyon balansingiz: {wallet['balance']}</b>\n"
408             "Chekni shu chatga yuborsangiz butun summa kartadan hisoblanadi. Hamyondan "
409             f"aniq summa ajratib, qolganini kartadan to'lash uchun xaridni <b>Mini Appda</b> davom ettiring."
410         )
411 except Exception:
412     logger.exception("Hamyon balansini o'qishda xato (user=%s)", user.get("id"))
413 _checkout_sessions[int(user["telegram_id"])] = course["id"]
414 await send_tg_message(
415     client,
416     chat_id,
417     f"📁 <b>To'lovni amalga oshirish</b>\n\n"
418     f"📄 Kurs: <b>{_escape(course['title'])}</b>\n"
419     f"💰 {price_block}{wallet_line}\n\n"
420     f"🏦 Bank: <b>{_escape(card['bank_name'])}</b>\n"
421     f"📇 Karta raqami (nusxalash uchun bosing):\n"
422     f"<code>{_escape(card['card_number'])}</code>\n"
423     f"👤 Qabul qiluvchi: <b>{_escape(card['card_holder'])}</b>\n\n"
424     "To'lovni amalga oshiring va <b>chek skrinshotini rasm ko'rinishida shu chatga yuboring</b>.\n"
425     "Admin tekshirishi bilan darslar sizga ochiladi.",
426     {"inline_keyboard": [
427         [{"text": "📄 Mini Appda ko'rish", "web_app": {"url": f"{settings.WEBAPP_URL}#course_{course['id']}"}}],
428         [{"text": "❌ Bekor qilish", "callback_data": "checkout:cancel"}]
429     ]},
430 )
431
432
433
434 async def _notify_admins_about_receipt(
435     client: httpx.AsyncClient, purchase: Dict[str, Any], photo_file_id: str
436 ) -> int:
437     """Yangi chekni barcha superadminlarga yuboradi."""
438     username = str(purchase.get("username") or "").rstrip("@")
439     username_label = f"@{_escape(username)}" if username else "username yo'q"
440     caption = (
441         f"📄 <b>YANGI TO'LOV CHEKI KELDI!</b>\n\n"
442         f"👤 <b>Talaba:</b> {_escape(purchase.get('student_name') or 'Talaba')} ({username_label})\n"
443         f"📄 <b>Telegram ID:</b> <code>{purchase.get('telegram_id') or 0}</code>\n"
444         f"📄 <b>Kurs:</b> {_escape(purchase.get('course_title') or 'Kurs')}\n"
445         f"💰 <b>Summa:</b> {_uzs(purchase.get('amount'))}\n"
446         f"📁 <b>To'lov turi:</b> {_escape(purchase.get('payment_method') or 'card').upper()}\n"
447         f"📄 <b>Buyurtma ID:</b> <code>{_escape(purchase.get('transaction_id'))}</code>\n\n"
448         "Chekni tekshirib, tugma orqali qaror qabul qiling."
449     )
450     keyboard = {
451         "inline_keyboard": [
452             [
453                 {"text": "✅ Tasdiqlash", "callback_data": f"approve_{purchase['transaction_id']}"},
454                 {"text": "❌ Rad etish", "callback_data": f"reject_{purchase['transaction_id']}"},
455             ],
456             [{"text": "📄 Admin panel", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}},
457         ]
458     }
459     notified = 0
460     for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
461         result = await send_tg_photo(client, admin_id, photo_file_id, caption, keyboard)
462         if result.get("ok"):
463             notified += 1
464     return notified
465
466
467 async def _store_telegram_receipt(
468     client: httpx.AsyncClient, file_id: str, transaction_id: str
469 ) -> Optional[str]:
470     """Telegramdan kelgan chekni R2 ga ko'chiradi, admin paneli ham rasmni ko'ra olishi uchun."""
471     try:
472         file_data = await _telegram_call(client, "getFile", {"file_id": file_id})
473         file_path = (file_data.get("result") or {}).get("file_path")
474         if not file_path:
475             return None
476         response = await client.get(f"https://api.telegram.org/file/bot{[REDACTED_BOT_TOKEN]}/{file_path}")
477         image_bytes = response.content
478         if response.status_code != 200 or not image_bytes or len(image_bytes) > 8 * 1024 * 1024:

```

```

479         return None
480     ext = str(file_path).rsplit(".", 1)[-1].lower() if "." in str(file_path) else "jpg"
481     if ext not in {"jpg", "jpeg", "png", "webp"}:
482         ext = "jpg"
483     content_type = "image/jpeg" if ext in {"jpg", "jpeg"} else f"image/{ext}"
484     return await asyncio.to_thread(
485         r2_client.upload_bytes, f"receipts/{transaction_id}.{ext}", image_bytes, content_type
486     ) or None
487 except (httpx.HTTPError, ValueError) as exc:
488     logger.warning("Telegram cheki R2 ga ko'chirilmadi: %s", exc)
489     return None
490
491
492 async def _handle_receipt_photo(client: httpx.AsyncClient, message: Dict[str, Any], user: Dict[str, Any]) -> bool:
493     telegram_id = int(user.get("telegram_id") or 0)
494     course_id = _checkout_sessions.get(telegram_id)
495     photos = message.get("photo") or []
496     if not course_id or not photos:
497         return False
498
499     store = get_store()
500     course = await store.get_course(course_id)
501     if not course:
502         _checkout_sessions.pop(telegram_id, None)
503         await send_tg_message(client, telegram_id, "⚠️ Kurs topilmadi. Iltimos, katalogdan qayta tanlang.")
504         return True
505
506     file_id = photos[-1].get("file_id")
507     if not file_id:
508         await send_tg_message(client, telegram_id, "⚠️ Rasmni o'qib bo'lmadi. Iltimos, yana yuboring.")
509         return True
510
511     # Foydalanuvchi captioni: uzunligini cheklab, tizim teglarini (SYSTEM[...],
512     # 'wallet:', 'promo:') soxtalashtirib bo'lmas qilamiz – rad etilganda hamyon
513     # qaytarish oqimi shu commentga qaram bo'lgani uchun uni tozalash muhim.
514     raw_caption = str(message.get("caption") or "").strip()[:300]
515     clean_caption = (
516         raw_caption.replace("SYSTEM[", "SYSTEM_[").replace("wallet:", "wallet_").replace("promo:", "promo_")
517         or None
518     )
519
520     transaction_id = f"rcp_{uuid.uuid4().hex[:12]}"
521     receipt_url = await _store_telegram_receipt(client, file_id, transaction_id)
522     # Botdan kelgan chek ham mini-app bilan bir xil chegirmali narxda qayd etiladi
523     try:
524         pricing = await course_pricing(store, course)
525     except Exception:
526         pricing = {"final_price": course.get("price")}
527     purchase = await store.create_purchase(
528         {
529             "user_id": user["id"],
530             "course_id": course["id"],
531             "course_title": course["title"],
532             "amount": pricing["final_price"],
533             "status": "pending_approval",
534             "payment_method": "telegram_receipt",
535             "transaction_id": transaction_id,
536             "telegram_id": telegram_id,
537             "student_name": user.get("name") or "Talaba",
538             "username": user.get("username"),
539             # Telegram file_id boshqa chatga sendPhoto qilish uchun yetarli; chekning o'zi Telegramda qoladi.
540             "receipt_image_url": receipt_url or f"tg-file:{file_id}",
541             "comment": clean_caption,
542         }
543     )
544     _checkout_sessions.pop(telegram_id, None)
545     notified = await _notify_admins_about_receipt(client, purchase, file_id)
546     await send_tg_message(
547         client,
548         telegram_id,
549         "✅ <b>Chekingiz qabul qilindi!</b>\n\n"
550         f"<b>{_escape(course['title'])}</b> uchun to'lovigiz tekshiruvga yuborildi. "
551         "Admin tasdiqlashi bilan kurs va yopiq kanal havolasi avtomatik yuboriladi.",
552     )
553     if not notified:
554         logger.warning("Chek adminlarga yuborilmadi: %s", purchase["transaction_id"])
555     return True
556
557
558 async def _get_effective_target_group(store, course_id: Optional[str] = None) -> Tuple[Optional[str], str]:
559     """Hozirda faol bo'lgan Target Guruh ID va nomini olish."""

```

```

560     if course_id:
561         course = await store.get_course(course_id)
562         if course and course.get("telegram_channel_id"):
563             return str(course["telegram_channel_id"]), str(course.get("title") or "Kurs Guruhi")
564
565     dyn_id = await store.get_setting("target_group_id")
566     dyn_title = await store.get_setting("target_group_title") or "Kurs Guruhi"
567     if dyn_id:
568         return str(dyn_id), str(dyn_title)
569
570     cfg_id = getattr(settings, "TARGET_GROUP_ID", "")
571     if cfg_id:
572         return str(cfg_id), "Target Guruh"
573
574     return None, ""
575
576
577 async def publish_lesson_to_group(
578     client: httpx.AsyncClient,
579     lesson_id: str | int,
580     admin_chat_id: int,
581 ) -> bool:
582     """Darsni Target Supergroupga faqat inline play tugmasi bilan chiqarish."""
583     store = get_store()
584     lesson = await store.get_lesson(lesson_id)
585     if not lesson:
586         courses = await store.list_courses(published_only=False)
587         for c in courses:
588             from app.api.courses import resolve_course_modules
589             for m in resolve_course_modules(c):
590                 for l in m.get("lessons", []):
591                     if str(l.get("id")) == str(lesson_id):
592                         lesson = {**l, "course_id": c["id"]}
593                         break
594                 if lesson:
595                     break
596         if lesson:
597             break
598
599     if not lesson:
600         await send_tg_message(client, admin_chat_id, f"❌ #{lesson_id} dars topilmadi.")
601         return False
602
603     target_id, target_title = await _get_effective_target_group(store, lesson.get("course_id"))
604     if not target_id:
605         await send_tg_message(
606             client,
607             admin_chat_id,
608             "❌ <b>Xatolik:</b> Darslar chiqariladigan Target Guruh hali belgilanmagan!\n\n"
609             "Botni kurs guruhingizga admin qilib qo'shing va o'sha guruhda <code>set_group</code> buyrug'ini yuboring.",
610         )
611         return False
612
613     lesson_title = lesson.get("title") or "Dars"
614     post_text = (
615         f"👉 <b>{_escape(lesson_title)}</b>\n\n"
616         "Videonni ko'rish uchun pastdagi tugmani bosing."
617     )
618     inline_keyboard = {
619         "inline_keyboard": [
620             [
621                 {
622                     "text": f"👉 {_escape(lesson_title[:28])}ni ko'rish",
623                     "callback_data": f"lesson:{lesson_id}",
624                 }
625             ]
626         ]
627     }
628
629     tg_target = int(target_id) if str(target_id).lstrip("-").isdigit() else target_id
630     res = await send_tg_message(
631         client,
632         tg_target,
633         post_text,
634         reply_markup=inline_keyboard,
635         protect_content=getattr(settings, "PROTECT_CONTENT", True),
636     )
637     if res.get("ok"):
638         msg_id = (res.get("result") or {}).get("message_id")
639         try:
640             await store.update_lesson(lesson_id, {"published_message_id": msg_id, "published": True})

```



```

641         except Exception:
642             pass
643         await send_tg_message(
644             client,
645             admin_chat_id,
646             f"✅ <b>{_escape(lesson_title)}</b> darsi <b>{_escape(target_title)}</b> guruhiga chiqarildi!\n"
647             f"Xabar ID: <code>{msg_id}</code>",
648         )
649         return True
650     else:
651         err = res.get("description") or "Noma'lum xatolik"
652         await send_tg_message(
653             client,
654             admin_chat_id,
655             f"❌ Guruhga chiqarishda xatolik yuz berdi: <i>{_escape(err)}</i>\n\n"
656             "Bot guruhda administrator ekanligini va xabar yozish huquqi borligini tekshiring.",
657         )
658         return False
659
660
661 async def handle_student_ephemeral_lesson(
662     client: httpx.AsyncClient,
663     callback_query: Dict[str, Any],
664     lesson_id: str | int,
665 ) -> None:
666     """Student uchun Ephemeral Protected Video jo'natish logikasi va cheklovlar nazorati."""
667     store = get_store()
668     query_id = callback_query.get("id", "")
669     tg_user = callback_query.get("from") or {}
670     user_id = int(tg_user.get("id") or 0)
671     chat = (callback_query.get("message") or {}).get("chat") or {}
672     chat_id = chat.get("id") or user_id
673
674     # 1. FOYDALANUVCHINING CHEKLANGANLIGINI TEKSHIRISH (Permissions / Blocklist)
675     is_blocked, reason = await store.is_user_blocked_for_lesson(user_id, lesson_id)
676     if is_blocked:
677         block_msg = "⚠️ Administrator tomonidan ushbu darsdan foydalanish cheklangan."
678         if reason:
679             block_msg += f"\nSabab: {reason}"
680         await _answer_callback(client, query_id, block_msg)
681         return
682
683     # 2. Rate limiting (1 student + 1 lesson = 5 soniyada 1 marta)
684     now = time.time()
685     rate_key = (user_id, str(lesson_id))
686     last_click = _user_lesson_rate_limits.get(rate_key, 0.0)
687     if now - last_click < 5.0:
688         await _answer_callback(client, query_id, "Iltimos, biroz kuting (5 soniya).")
689         return
690     _user_lesson_rate_limits[rate_key] = now
691
692     # Xotira tozalash
693     if len(_user_lesson_rate_limits) > 1000:
694         cutoff = now - 3600.0
695         for k, ts in list(_user_lesson_rate_limits.items()):
696             if ts < cutoff:
697                 _user_lesson_rate_limits.pop(k, None)
698
699     # 3. Darsni bazadan olish
700     lesson = await store.get_lesson(lesson_id)
701     target_course_id = (lesson or {}).get("course_id")
702     if not lesson:
703         courses = await store.list_courses(published_only=False)
704         for c in courses:
705             from app.api.courses import resolve_course_modules
706             for m in resolve_course_modules(c):
707                 for l in m.get("lessons", []):
708                     if str(l.get("id")) == str(lesson_id):
709                         lesson = {**l, "course_id": c["id"]}
710                         target_course_id = c["id"]
711                         break
712                 if lesson:
713                     break
714             if lesson:
715                 break
716
717     if not lesson:
718         await _answer_callback(client, query_id, "Ushbu dars topilmadi yoki o'chirilgan.")
719         return
720
721     # 4. Huquq tekshiruvi: Admin, Preview yoki Xarid qilgan talaba

```

```

722     authorized = False
723     if is_admin(user_id):
724         authorized = True
725     elif lesson.get("is_preview"):
726         authorized = True
727     else:
728         user = await store.get_user_by_tg(user_id)
729         if user:
730             if target_course_id:
731                 enr = await store.get_enrollment(user["id"], target_course_id)
732                 if enr and enr.get("status") == "active":
733                     authorized = True
734             else:
735                 purchase = await store.get_approved_purchase_for(user["id"], target_course_id)
736                 if purchase:
737                     authorized = True
738                     await store.create_enrollment(user["id"], target_course_id, purchase.get("id"))
739             else:
740                 enrs = await store.list_enrollments(user["id"])
741                 if enrs:
742                     authorized = True
743         if not authorized:
744             all_approved = await store.list_purchases(status="approved")
745             for p in all_approved:
746                 if p.get("telegram_id") == user_id:
747                     if not target_course_id or p.get("course_id") == target_course_id:
748                         authorized = True
749                         break
750
751     if not authorized:
752         await _answer_callback(
753             client,
754             query_id,
755             "🚫 Bu darsni ko'rish uchun kursga yozilgan (xarid qilgan) bo'lishingiz kerak. Mini App orqali kursni xarid qiling.",
756         )
757     return
758
759 # 5. Video faylini aniqlash
760 video_file_id = lesson.get("video_file_id") or lesson.get("telegram_file_id") or lesson.get("file_id")
761 if not video_file_id and str(lesson.get("video_url") or "").startswith("tg-file:"):
762     video_file_id = str(lesson["video_url"]).split("tg-file:", 1)[1]
763
764 if not video_file_id:
765     await _answer_callback(client, query_id, "Ushbu dars uchun video fayli topilmadi.")
766     return
767
768 title = lesson.get("title") or "Dars"
769 caption = f"📺 <b>{_escape(title)}</b>"
770 duration = lesson.get("duration_seconds")
771 if duration is None and isinstance(lesson.get("duration"), int):
772     duration = lesson["duration"]
773
774 # 6. Telegram Ephemeral Protected Video yuborish
775 res = await send_ephemeral_video(
776     client=client,
777     chat_id=chat_id,
778     video_file_id=video_file_id,
779     caption=caption,
780     callback_user_id=user_id,
781     callback_query_id=query_id,
782     replace_callback_query_message=False,
783     duration=duration,
784     width=lesson.get("width"),
785     height=lesson.get("height"),
786     protect_content=getattr(settings, "PROTECT_CONTENT", True),
787 )
788
789 if not res.get("ok"):
790     logger.warning("Ephemeral video yuborilmadi: %s. Direct fallback ishlatilmoqda...", res)
791     # Fallback: agar chat supergroup bo'lmasa yoki ephemeral qo'llab-quvvatlanmasa, privat chatga yuboramiz
792     direct_res = await send_tg_video(
793         client=client,
794         chat_id=user_id,
795         video=video_file_id,
796         caption=f"{caption}\n\n🔒 <i>Himoyalangan dars videosi (faqat siz uchun)</i>",
797         protect_content=True,
798         supports_streaming=True,
799         duration=duration,
800         width=lesson.get("width"),
801         height=lesson.get("height"),

```

```

802     )
803     if direct_res.get("ok"):
804         await _answer_callback(client, query_id, "✅ Video shaxsiy bot chatiga yuborildi!")
805     else:
806         await _answer_callback(client, query_id, "❌ Video ochilmadi. Iltimos botga /start bosing.")
807     return
808 else:
809     await _answer_callback(client, query_id, "📺 Video ochildi")
810
811 # 7. Telemetriya & o'zlashtirish progressi
812 try:
813     await store.log_access({
814         "lesson_id": str(lesson.get("id") or lesson_id),
815         "user_id": user_id,
816         "username": tg_user.get("username"),
817         "first_name": tg_user.get("first_name"),
818         "client_callback_id": query_id,
819         "mode": "ephemeral",
820     })
821 except Exception as exc:
822     logger.debug("Access log yozilmadi: %s", exc)
823
824 user = await store.get_user_by_tg(user_id)
825 if user and target_course_id:
826     try:
827         await store.upsert_progress(user["id"], target_course_id, str(lesson.get("id") or lesson_id), completed=False)
828     except Exception as exc:
829         logger.debug("Progress yangilanmadi: %s", exc)
830
831
832 async def _handle_admin_video_message(
833     client: httpx.AsyncClient,
834     message: Dict[str, Any],
835     admin_id: int,
836 ) -> bool:
837     """Admin to'g'ridan-to'g'ri video yuborganida 1-bosishda dars yaratish oqimi."""
838     video = message.get("video")
839     if not video and message.get("document") and str((message.get("document") or {}).get("mime_type") or "").startswith("video/"):
840         video = message.get("document")
841
842     if not video or not is_admin(admin_id):
843         return False
844
845     store = get_store()
846     file_id = video.get("file_id")
847     file_unique_id = video.get("file_unique_id")
848     duration = int(video.get("duration") or 0)
849     width = video.get("width")
850     height = video.get("height")
851     file_size = int(video.get("file_size") or 0)
852     caption = str(message.get("caption") or "").strip()
853
854     courses = await store.list_courses(published_only=True)
855     active_course = courses[0] if courses else None
856     course_id = active_course["id"] if active_course else None
857
858     existing_count = await store.count_lessons(course_id=course_id)
859     if existing_count == 0 and active_course:
860         from app.api.courses import resolve_course_modules
861         existing_count = len([l for m in resolve_course_modules(active_course) for l in m.get("lessons", [])])
862     next_num = existing_count + 1
863
864     if caption:
865         if "dars" in caption.lower():
866             suggested_title = caption
867         else:
868             suggested_title = f"{next_num}-dars. {caption}"
869     else:
870         suggested_title = f"{next_num}-dars"
871
872     _pending_admin_videos[admin_id] = {
873         "file_id": file_id,
874         "video_file_id": file_id,
875         "video_file_unique_id": file_unique_id,
876         "duration": duration,
877         "duration_seconds": duration,
878         "width": width,
879         "height": height,
880         "file_size": file_size,
881         "suggested_title": suggested_title,
882         "course_id": course_id,

```

```

883         "created_by": admin_id,
884     }
885
886     dur_str = f"{duration // 60:02d}:{duration % 60:02d}"
887     size_mb = round(file_size / (1024 * 1024), 1)
888
889     text = (
890         "📺 <b>Yangi video dars qabul qilindi!</b>\n\n"
891         f"🕒 <b>Davomiyligi:</b> <code>{dur_str}</code> | 📄 <b>Hajmi:</b> <code>{size_mb} MB</code>\n\n"
892         f"💡 <b>Taklif etilgan nom:</b>\n"
893         f"👉 <b>{<_escape(suggested_title)></b>\n\n"
894         "<i>Quyidagi tugmani 1 marta bosib darsni darhol saqlashingiz mumkin, "
895         "yoki boshqa nom xohlasangiz, uni shunchaki yozib yuboring:</i>"
896     )
897
898     keyboard = {
899         "inline_keyboard": [
900             [{"text": f"✅ «{suggested_title[:28]}» deb saqlash", "callback_data": "admin:save_video"}],
901             [{"text": f"❌ Bekor qilish", "callback_data": "admin:cancel_video"}],
902         ]
903     }
904
905     await send_tg_message(client, admin_id, text, reply_markup=keyboard)
906     return True
907
908
909 async def _save_pending_admin_video(
910     client: httpx.AsyncClient,
911     admin_id: int,
912     custom_title: Optional[str] = None,
913     message_id_to_edit: Optional[int] = None,
914 ) -> Optional[Dict[str, Any]]:
915     """Kutilayotgan videoni bazaga va Mini App katalogiga sinxron saqlash."""
916     video_data = _pending_admin_videos.pop(admin_id, None)
917     if not video_data:
918         return None
919
920     store = get_store()
921     title = (custom_title or video_data.get("suggested_title") or "Yangi dars").strip()
922     course_id = video_data.get("course_id")
923     if not course_id:
924         courses = await store.list_courses(published_only=True)
925         if courses:
926             course_id = courses[0]["id"]
927
928     dur_sec = int(video_data.get("duration") or 0)
929     dur_str = f"{dur_sec // 60:02d}:{dur_sec % 60:02d}"
930     size_mb = round((video_data.get("file_size") or 0) / (1024 * 1024), 1)
931
932     lesson = await store.add_lesson({
933         "title": title,
934         "course_id": course_id,
935         "video_file_id": video_data.get("video_file_id"),
936         "video_file_unique_id": video_data.get("video_file_unique_id"),
937         "duration": dur_str,
938         "duration_seconds": dur_sec,
939         "width": video_data.get("width"),
940         "height": video_data.get("height"),
941         "file_size": video_data.get("file_size"),
942         "created_by": admin_id,
943     })
944
945     lesson_id = lesson["id"]
946
947     # Mini App katalogiga va modullariga sinxronlash
948     if course_id:
949         course = await store.get_course(course_id)
950         if course:
951             modules = list(course.get("modules") or [])
952             if not modules:
953                 modules = [{"title": "01. Asosiy Video Darslar", "lessons": []}]
954             modules[-1].setdefault("lessons", []).append({
955                 "id": str(lesson_id),
956                 "title": title,
957                 "duration": dur_str,
958                 "video_url": f"tg-file:{video_data.get('video_file_id')}",
959                 "telegram_file_id": video_data.get("video_file_id"),
960                 "file_id": video_data.get("video_file_id"),
961                 "is_preview": False,
962             })
963             course["modules"] = modules

```

```

964         course["lesson_count"] = sum(len(m.get("lessons", [])) for m in modules)
965         await store.upsert_course(course)
966
967     target_id, target_title = await _get_effective_target_group(store, course_id)
968     target_name = target_title if target_id else "Belgilanmagan (/set_group)"
969
970     confirmation_text = (
971         f"📌 <b>{_escape(title)}</b> muvaffaqiyatli saqlandi!\n\n"
972         f"📌 <b>Dars ID:</b> <code>{lesson_id}</code>\n"
973         f"🕒 <b>Davomiyligi:</b> <code>{dur_str}</code> | 📄 <b>Hajmi:</b> <code>{size_mb} MB</code>\n"
974         f"📌 <b>Target Guruh:</b> <b>{_escape(target_name)}</b>\n\n"
975         "Endi darsni guruhga chiqarishingiz yoki yangi video yuklashingiz mumkin:"
976     )
977
978     keyboard = {
979         "inline_keyboard": [
980             [{"text": "📌 Guruhga chiqarish", "callback_data": f"admin:publish:{lesson_id}"}],
981             [
982                 {"text": "📎 Yana dars qo'shish", "callback_data": "admin:add_video"},
983                 {"text": "📄 Darslar ro'yxati", "callback_data": "admin:lessons"}
984             ],
985             [{"text": "⚙️ Admin Panel (Mini App)", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}],
986         ]
987     }
988
989     if message_id_to_edit:
990         edit_res = await _telegram_call(
991             client,
992             "editMessageText",
993             {"chat_id": admin_id, "message_id": message_id_to_edit, "text": confirmation_text, "parse_mode": "HTML", "reply_markup": keyboard}
994         )
995         if not edit_res.get("ok"):
996             await send_tg_message(client, admin_id, confirmation_text, reply_markup=keyboard)
997     else:
998         await send_tg_message(client, admin_id, confirmation_text, reply_markup=keyboard)
999
1000     return lesson
1001
1002
1003 async def _send_admin_lessons_list(client: httpx.AsyncClient, admin_id: int) -> None:
1004     """Admin uchun barcha darslar va ularni guruhga chiqarish tugmalari."""
1005     store = get_store()
1006     lessons = await store.list_lessons()
1007     if not lessons:
1008         await send_tg_message(
1009             client,
1010             admin_id,
1011             f"📌 Hozircha saqlangan video darslar yo'q.\n\nVideo yuboring va 1-bosishda dars yarating!",
1012             {"inline_keyboard": [{"text": "📎 Video yuklash", "callback_data": "admin:add_video"}]}
1013         )
1014     return
1015
1016     text = f"📄 <b>Barcha Video Darslar ({len(lessons)} ta):</b>\n\n"
1017     rows = []
1018     for l in lessons[:15]:
1019         lid = l["id"]
1020         ltitle = l.get("title") or "Dars"
1021         dur = l.get("duration") or ""
1022         text += f"• <b>{_escape(ltitle)}</b> (<code>{dur}</code>) – ID: <code>{lid}</code>\n"
1023         rows.append([
1024             {"text": f"📌 #{lid} Guruhga chiqarish", "callback_data": f"admin:publish:{lid}"},
1025             {"text": f"📄 Sinab ko'rish", "callback_data": f"lesson:{lid}"},
1026         ])
1027
1028     rows.append([{"text": "📎 Yangi dars qo'shish", "callback_data": "admin:add_video"}])
1029     await send_tg_message(client, admin_id, text, reply_markup={"inline_keyboard": rows})
1030
1031
1032 async def _send_direct_lesson_video(
1033     client: httpx.AsyncClient,
1034     chat_id: int,
1035     user_id: int,
1036     lesson_id: str | int,
1037 ) -> None:
1038     """Talabaga dars videosini to'g'ridan-to'g'ri shaxsiy chatda himoyalangan holda yuborish."""
1039     store = get_store()
1040     lesson = await store.get_lesson(lesson_id)
1041     target_course_id = (lesson or {}).get("course_id")
1042     if not lesson:
1043         courses = await store.list_courses(published_only=False)
1044         for c in courses:

```

```

1045         from app.api.courses import resolve_course_modules
1046         for m in resolve_course_modules(c):
1047             for l in m.get("lessons", []):
1048                 if str(l.get("id")) == str(lesson_id):
1049                     lesson = {**l, "course_id": c["id"]}
1050                     target_course_id = c["id"]
1051                     break
1052             if lesson:
1053                 break
1054         if lesson:
1055             break
1056
1057     if not lesson:
1058         await send_tg_message(client, chat_id, "❌ Ushbu dars topilmadi.")
1059         return
1060
1061     authorized = False
1062     if is_admin(user_id) or lesson.get("is_preview"):
1063         authorized = True
1064     else:
1065         user = await store.get_user_by_tg(user_id)
1066         if user and target_course_id:
1067             enr = await store.get_enrollment(user["id"], target_course_id)
1068             if enr and enr.get("status") == "active":
1069                 authorized = True
1070
1071     if not authorized:
1072         await send_tg_message(
1073             client,
1074             chat_id,
1075             f"🔒 <b>{_escape(lesson.get('title') or 'Dars')}</b> himoyalangan dars.\n\n"
1076             "Ushbu darsni ko'rish uchun avval kursni xarid qilishingiz kerak.",
1077             {"inline_keyboard": [{"text": "🛒 Mini Appda xarid qilish", "web_app": {"url": settings.WEBAPP_URL}}]}
1078         )
1079         return
1080
1081     video_file_id = lesson.get("video_file_id") or lesson.get("telegram_file_id") or lesson.get("file_id")
1082     if not video_file_id and str(lesson.get("video_url") or "").startswith("tg-file:"):
1083         video_file_id = str(lesson["video_url"]).split("tg-file:", 1)[1]
1084
1085     if not video_file_id:
1086         await send_tg_message(client, chat_id, "❌ Video fayli topilmadi.")
1087         return
1088
1089     title = lesson.get("title") or "Dars"
1090     caption = f"📺 <b>{_escape(title)}</b>\n\n📺 <i>Himoyalangan dars videosi</i>"
1091     duration = lesson.get("duration_seconds")
1092     if duration is None and isinstance(lesson.get("duration"), int):
1093         duration = lesson["duration"]
1094
1095     res = await send_tg_video(
1096         client=client,
1097         chat_id=chat_id,
1098         video=video_file_id,
1099         caption=caption,
1100         protect_content=getattr(settings, "PROTECT_CONTENT", True),
1101         supports_streaming=True,
1102         duration=duration,
1103         width=lesson.get("width"),
1104         height=lesson.get("height"),
1105     )
1106     if not res.get("ok"):
1107         await send_tg_message(client, chat_id, "❌ Video yuborishda xatolik yuz berdi.")
1108
1109
1110 async def _handle_admin_callback(client: httpx.AsyncClient, callback_query: Dict[str, Any]) -> bool:
1111     data = str(callback_query.get("data") or "")
1112     query_id = callback_query.get("id", "")
1113     admin_tg_id = callback_query.get("from", {}).get("id")
1114
1115     # Admin video & darslar boshqaruvi
1116     if data.startswith("admin:"):
1117         if not is_admin(admin_tg_id):
1118             await _answer_callback(client, query_id, "Sizda admin huquqlari yo'q")
1119             return True
1120
1121         if data == "admin:save_video":
1122             msg = callback_query.get("message") or {}
1123             msg_id = msg.get("message_id")
1124             await _answer_callback(client, query_id, "✅ Dars saqlanmoqda...")
1125             await _save_pending_admin_video(client, int(admin_tg_id), message_id_to_edit=msg_id)

```

```

1126         return True
1127
1128     if data == "admin:cancel_video":
1129         _pending_admin_videos.pop(int(admin_tg_id), None)
1130         await _answer_callback(client, query_id, "Video bekor qilindi.")
1131         msg = callback_query.get("message") or {}
1132         msg_id = msg.get("message_id")
1133         if msg_id:
1134             await _telegram_call(
1135                 client,
1136                 "editMessageText",
1137                 {"chat_id": admin_tg_id, "message_id": msg_id, "text": "❌ Video yuklash bekor qilindi.", "parse_mode": "HTML"}
1138             )
1139         return True
1140
1141     if data.startswith("admin:publish:"):
1142         lid = data.split(":", 2)[2]
1143         await _answer_callback(client, query_id, "Guruhga chiqarilmoqda...")
1144         await publish_lesson_to_group(client, lid, int(admin_tg_id))
1145         return True
1146
1147     if data == "admin:lessons":
1148         await _answer_callback(client, query_id)
1149         await _send_admin_lessons_list(client, int(admin_tg_id))
1150         return True
1151
1152     if data == "admin:add_video":
1153         await _answer_callback(client, query_id)
1154         await send_tg_message(
1155             client,
1156             int(admin_tg_id),
1157             "📺 <b>Yangi dars videosini yuboring.</b>\n\n"
1158             "Video yuborganingizda tizim uning hajmi va davomiyligini tekshirib, 1-bosishda dars yaratish imkonini beradi."
1159         )
1160         return True
1161
1162     return False
1163
1164     if not (data.startswith("approve_") or data.startswith("reject_")):
1165         return False
1166
1167     if admin_tg_id not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1168         await _answer_callback(client, query_id, "Sizda admin huquqlari yo'q")
1169         return True
1170
1171     order_id = data.split("_", 1)[1]
1172     profile = settings.ADMIN_PROFILES.get(admin_tg_id, {})
1173     admin_name = profile.get("name") or callback_query.get("from", {}).get("first_name") or "Superadmin"
1174     approved = data.startswith("approve_")
1175     ok, result = (
1176         await approve_purchase(order_id, admin_name)
1177         if approved
1178         else await reject_purchase(order_id, admin_name)
1179     )
1180     await _answer_callback(client, query_id, ("✅ " if ok else "⚠️ ") + result)
1181     if not ok:
1182         return True
1183
1184     message = callback_query.get("message") or {}
1185     chat = message.get("chat") or {}
1186     current_text = str(message.get("caption") or message.get("text") or "")
1187     status_line = "✅ <b>TO'LOV TASDIQLANDI</b>" if approved else "❌ <b>TO'LOV RAD ETILDI</b>"
1188     # Telegram eski captionni tekis matn (entities qo'llanmagan holda) qaytaradi.
1189     # Uni parse_mode=HTML bilan qayta yuborishdan oldin escape qilish shart –
1190     # talaba ismi/izohidagi & yoki < "can't parse entities" xatosini keltirib,
1191     # chek kartasiga status qatori umuman qo'shilmay qolardi.
1192     updated = (
1193         f"_{escape(current_text)}\n\n—————\n"
1194         f"_{status_line}\n"
1195         f"👤 <b>Admin:</b> {escape(admin_name)}"
1196     )[:1024]
1197     method = "editMessageCaption" if message.get("caption") is not None else "editMessageText"
1198     field = "caption" if method == "editMessageCaption" else "text"
1199     edit_result = await _telegram_call(
1200         client,
1201         method,
1202         {"chat_id": chat.get("id"), "message_id": message.get("message_id"), field: updated, "parse_mode": "HTML"},
1203     )
1204     if not edit_result.get("ok"):
1205         # Tahrir o'tmasa (masalan, caption 1024 belgi chekkasida kesilganda ham
1206         # xato qaytishi mumkin) – qarorni alohida xabar sifatida yozamiz.

```

```

1207         await send_tg_message(
1208             client,
1209             int(chat.get("id") or 0) or int(admin_tg_id or 0),
1210             f"{status_line}\n"
1211             f"👤 <b>Admin:</b> { _escape(admin_name)}\n"
1212             f"🛒 <b>Buyurtma:</b> <code>{_escape(order_id)}</code>",
1213         )
1214     return True
1215
1216
1217 async def handle_callback_query(client: httpx.AsyncClient, callback_query: Dict[str, Any]) -> None:
1218     if await _handle_admin_callback(client, callback_query):
1219         return
1220
1221     query_id = callback_query.get("id", "")
1222     data = str(callback_query.get("data") or "")
1223     tg_user = callback_query.get("from") or {}
1224     chat_id = (callback_query.get("message") or {}).get("chat", {}).get("id") or tg_user.get("id")
1225     user = await _ensure_user(tg_user)
1226     if not user or not chat_id:
1227         await _answer_callback(client, query_id, "Foydalanuvchi aniqlanmadi")
1228         return
1229
1230     if data.startswith("lesson:"):
1231         lesson_id = data.split(":", 1)[1]
1232         await handle_student_ephemeral_lesson(client, callback_query, lesson_id)
1233         return
1234
1235     if data == "noop":
1236         await _answer_callback(client, query_id)
1237         return
1238     if data.startswith("course:"):
1239         try:
1240             index = int(data.split(":", 1)[1])
1241         except ValueError:
1242             await _answer_callback(client, query_id, "Kurs topilmadi")
1243             return
1244         await _answer_callback(client, query_id)
1245         await show_course_card(client, int(chat_id), index)
1246         return
1247     if data.startswith("pay:"):
1248         await _answer_callback(client, query_id)
1249         await _start_checkout(client, int(chat_id), user, data.split(":", 1)[1])
1250         return
1251     if data == "checkout:cancel":
1252         _checkout_sessions.pop(int(user["telegram_id"]), None)
1253         await _answer_callback(client, query_id, "To'lov bekor qilindi")
1254         await send_tg_message(client, int(chat_id), "To'lov jarayoni bekor qilindi. Istalgan payt kursni qayta tanlashingiz mumkin.")
1255         return
1256     if data == "payments":
1257         await _answer_callback(client, query_id)
1258         await _send_payment_info(client, int(chat_id))
1259         return
1260     if data == "help":
1261         await _answer_callback(client, query_id)
1262         await _send_help(client, int(chat_id))
1263         return
1264     await _answer_callback(client, query_id, "Bu tugma endi faol emas")
1265
1266
1267 async def handle_chat_join_request(client: httpx.AsyncClient, join_request: Dict[str, Any]) -> None:
1268     """Link va xaridor akkaunti mos kelgandagina join requestni tasdiqlaydi.
1269
1270     Tasdiqlangach havola darhol bekor qilinadi (bir martalik link): uni boshqa
1271     odamga qayta yuborib yoki eski zavavkani qayta bosib kirib bo'lmaydi.
1272     """
1273     chat = join_request.get("chat") or {}
1274     requester = join_request.get("from") or {}
1275     chat_id = chat.get("id")
1276     telegram_id = requester.get("id")
1277     invite = join_request.get("invite_link") or {}
1278     invite_link = invite.get("invite_link") if isinstance(invite, dict) else None
1279     if not chat_id or not telegram_id:
1280         return
1281
1282     authorized, purchase = await is_join_request_authorized(int(chat_id), int(telegram_id), invite_link)
1283     method = "approveChatJoinRequest" if authorized else "declineChatJoinRequest"
1284     result = await _telegram_call(
1285         client, method, {"chat_id": chat_id, "user_id": telegram_id}
1286     )
1287     if not result.get("ok"):

```



```

1288         return
1289
1290     title = _escape((purchase or {}).get("course_title") or chat.get("title") or "kurs")
1291     if authorized:
1292         if invite_link:
1293             await revoke_join_request_link(client, int(chat_id), invite_link)
1294             await send_tg_message(
1295                 client,
1296                 int(telegram_id),
1297                 f"👋 <b>Xush kelibsiz!</b> Sizning <b>{title}</b> kanaliga zayavkangiz tasdiqlandi. Darslarni boshlashingiz mumkin.",
1298             )
1299         else:
1300             await send_tg_message(
1301                 client,
1302                 int(telegram_id),
1303                 "🚧 <b>Kanalga kirish rad etildi.</b> Bu havola boshqa akkauntga berilgan, allaqachon ishlatilgan yoki amal qilish muddati tugagan."
1304                 "Kursni o'z akkauntingizdan sotib oling.",
1305                 {"inline_keyboard": [{"text": "👉 Mini Appni ochish", "web_app": {"url": settings.WEBAPP_URL}}]},
1306             )
1307
1308
1309     async def handle_my_chat_member(client: httpx.AsyncClient, update: Dict[str, Any]) -> None:
1310         """Bot kanalga admin bo'lganda superadminlarga kanal ID sini yuboradi.
1311         Guruhga qo'shilganda esa qisqa tanishtiruv yuboradi."""
1312         chat = update.get("chat") or {}
1313         old_status = (update.get("old_chat_member") or {}).get("status")
1314         new_status = (update.get("new_chat_member") or {}).get("status")
1315
1316         if chat.get("type") in {"group", "supergroup"}:
1317             if new_status in {"member", "administrator"} and old_status in {"left", "kicked"}:
1318                 await _send_group_intro(client, int(chat.get("id") or 0))
1319                 return
1320
1321             if new_status not in {"administrator", "creator"} or old_status == new_status:
1322                 return
1323             if chat.get("type") not in {"channel", "supergroup"}:
1324                 return
1325
1326             text = (
1327                 "👋 <b>Yangi dars kanali ulandi</b>\n\n"
1328                 f"🌟 Nomi: <b>{_escape(chat.get('title') or 'Kanal')}</b>\n\n"
1329                 f"📄 Kanal ID: <code>{chat.get('id')}</code>\n\n"
1330                 "Admin paneldagi kurs tahrirlash oynasiga shu ID ni <code>telegram_channel_id</code> sifatida kiriting."
1331             )
1332             for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1333                 await send_tg_message(client, admin_id, text)
1334
1335
1336     async def _send_help(client: httpx.AsyncClient, chat_id: int) -> None:
1337         await send_tg_message(
1338             client,
1339             chat_id,
1340             "💡 <b>Kreativ AI – Yordam markazi</b>\n\n"
1341             "To'lov, kurslar, yopiq kanallar yoki takliflar bo'yicha superadminlarga yozing:\n\n"
1342             "👤 <b>Yaxshi Bola</b> – @yomonboIa\n"
1343             "👤 <b>Zuhra Olimova</b> – @sokin_notalar",
1344             {"inline_keyboard": [
1345                 [{"text": "👉 Kreativ AI Mini App", "web_app": {"url": settings.WEBAPP_URL}},
1346                 [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}]
1347             ]},
1348         )
1349
1350
1351     async def _send_welcome(client: httpx.AsyncClient, chat_id: int, tg_user: Dict[str, Any]) -> None:
1352         first_name = _escape(tg_user.get("first_name") or "Do'stim")
1353         is_admin = tg_user.get("id") in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]
1354         text = (
1355             f"Assalomu alaykum, <b>{first_name}</b>! 🌟\n\n"
1356             "<b>Kreativ AI</b> – Sun'iy intellekt, dizayn va zamonaviy kasblar platformasiga xush kelibsiz!\n\n"
1357             "🔥 <b>Imkoniyatlar:</b>\n\n"
1358             "• 📖 Yuqori sifatli amaliy kurslar\n"
1359             "• 📄 Tekshiriluvchi QR-kodli sertifikatlar\n"
1360             "• 🏠 Himoyalangan yopiq dars kanallari\n\n"
1361             "Darslarni boshlash uchun quyidagi tugmani bosing 📲"
1362         )
1363         keyboard_rows = [
1364             [{"text": "👉 Platformani Ochish (Mini App)", "web_app": {"url": settings.WEBAPP_URL}},
1365             [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}, {"text": "💰 To'lov Rekvizitlari", "callback_data": "payments"}],
1366             [{"text": "👤 Admin Yordami", "callback_data": "help"}],
1367         ]
1368         if is_admin:

```

```

1369         keyboard_rows.append([{"text": " 🌀 Superadmin Boshqaruv Paneli", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}])
1370     keyboard = {"inline_keyboard": keyboard_rows}
1371
1372     banner_url = getattr(settings, "WELCOME_BANNER_URL", "")
1373     if not banner_url:
1374         try:
1375             courses = await get_store().list_courses(published_only=True)
1376             banner_url = str(courses[0].get("cover_url") or "") if courses else ""
1377         except Exception:
1378             banner_url = ""
1379     banner_url = _resolve_media_url(banner_url)
1380     if banner_url.startswith(("https://", "http://")):
1381         result = await send_tg_photo(client, chat_id, banner_url, text, keyboard)
1382         if result.get("ok"):
1383             return
1384     await send_tg_message(client, chat_id, text, keyboard)
1385
1386
1387 async def _send_stats(client: httpx.AsyncClient, chat_id: int, user_id: int) -> None:
1388     if user_id not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1389         await send_tg_message(client, chat_id, " ❌ Bu buyruq faqat superadminlar uchun.")
1390         return
1391     store = get_store()
1392     stats = await store.revenue_stats()
1393     pending = await store.list_purchases(status="pending_approval", limit=1000)
1394     active_courses = await store.list_courses(published_only=True)
1395     # Ayrim adminlarga (masalan Zuhra Olimova) talabalar soni ko'rsatilmaydi
1396     students_line = ""
1397     if user_id not in settings.STATS_STUDENTS_HIDDEN_IDS:
1398         students_line = f" 🧑 Talabalar: <b>{await store.count_users()} ta</b>\n"
1399     text = (
1400         f" 📊 <b>Kreativ AI – Platforma Statistikasi</b>\n\n"
1401         f" 📈 Jami tushum: <b>{_uzs(stats.get('total_revenue'))}</b>\n"
1402         f" 📅 Oylik tushum: <b>{_uzs(stats.get('monthly_revenue'))}</b>\n"
1403         f"{students_line}"
1404         f" 📚 Faol kurslar: <b>{len(active_courses)} ta</b>\n"
1405         f" 🕒 Kutilayotgan cheklar: <b>{len(pending)} ta</b>"
1406     )
1407     await send_tg_message(
1408         client, chat_id, text,
1409         {"inline_keyboard": [{"text": " 🌀 Admin Panelni Ochish", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}]}],
1410     )
1411
1412
1413
1414 async def _handle_unknown_text(client: httpx.AsyncClient, chat_id: int) -> None:
1415     """Matn ko'rinishidagi barcha xabarlariga yo'naltiruvchi javob (AI chat olib tashlangan)."""
1416     await send_tg_message(
1417         client,
1418         chat_id,
1419         f" 📚 <b>Kurslar platformasi</b>\n\n"
1420         "Kurslarni ko'rish uchun katalogni oching yoki to'lov uchun rekvizitlarni so'rang. "
1421         "Savollaringiz bo'lsa, adminlar /help bo'limida.",
1422         {"inline_keyboard": [
1423             [{"text": " 📚 Kurslar Katalogi", "callback_data": "course:0"}, {"text": " 💰 To'lov Rekvizitlari", "callback_data": "payments"}],
1424             [{"text": " 📱 Mini App", "web_app": {"url": settings.WEBAPP_URL}}],
1425         ]},
1426     )
1427
1428
1429 async def _get_bot_id(client: httpx.AsyncClient) -> Optional[int]:
1430     """Botning o'z ID sini bir marta olib keshlaydi (guruhda reply-to-bot aniqlash uchun)."""
1431     global BOT_ID
1432     if BOT_ID is None:
1433         data = await _telegram_call(client, "getMe", {})
1434         BOT_ID = (data.get("result") or {}).get("id")
1435     return BOT_ID
1436
1437
1438 def _group_ai_allowed(chat_id: int) -> bool:
1439     """Har bir guruhda AI javoblari: kamida 15 soniya oraliq va soatiga 8 tadan."""
1440     now = time.time()
1441     st = _GROUP_AI_STATE.setdefault(chat_id, {"times": [], "last": 0.0})
1442     st["times"] = [t for t in st["times"] if now - t < 3600]
1443     if now - st["last"] < 15:
1444         return False
1445     if len(st["times"]) >= 8:
1446         return False
1447     return True
1448
1449

```

```

1450 def _group_ai_mark(chat_id: int) -> None:
1451     now = time.time()
1452     st = _GROUP_AI_STATE.setdefault(chat_id, {"times": [], "last": 0.0})
1453     st["last"] = now
1454     st["times"].append(now)
1455
1456
1457 async def _send_group_intro(client: httpx.AsyncClient, chat_id: int) -> None:
1458     """Bot guruhga qo'shilganda qisqa tanishtiruv."""
1459     if not chat_id:
1460         return
1461     text = (
1462         "👋 <b>Assalomu alaykum!</b> Men – <b>Kreativ AI</b> o'quv platformasining yordamchisman.\n\n"
1463         "📚 Guruhda savolingizni <b>@kreativaibot</b> deb yozib bering – AI yordamchim javob beradi "
1464         "va kasb o'rganish bo'yicha yo'l ko'rsatadi.\n\n"
1465         "📖 Kurslar, narxlar va chegirmalar haqida bemalol so'rang!"
1466     )
1467     keyboard = {
1468         "inline_keyboard": [
1469             [{"text": "📖 Kurslarni ko'rish (Mini App)", "web_app": {"url": settings.WEBAPP_URL}}],
1470             [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}],
1471         ]
1472     }
1473     await send_tg_message(client, chat_id, text, keyboard, protect_content=False)
1474
1475
1476 async def _build_catalog_context(store) -> str:
1477     """AI ga beriladigan katalog konteksti – faqat real ma'lumotlar (raqam o'ylab topish taqiqlanadi)."""
1478     try:
1479         courses = await store.list_courses(published_only=True)
1480     except Exception:
1481         courses = []
1482     lines: list = []
1483     for c in courses[:8]:
1484         try:
1485             pricing = await course_pricing(store, c)
1486         except Exception:
1487             pricing = {"discount_active": False, "discount_spots_left": None, "final_price": c.get("price")}
1488         line = (
1489             f"• «{c.get('title')}» – kategoriya: {c.get('category')}, "
1490             f"{int(c.get('lesson_count') or 0)} ta dars, narxi {_uzs(pricing['final_price'])}, "
1491             f"ustoz: {c.get('instructor_name')}"
1492         )
1493         if pricing["discount_active"] and c.get("discount_percent"):
1494             line += (
1495                 f", 🍷 -{int(c['discount_percent'])}% chegirma birinchi {int(c.get('discount_limit') or 0)} kishi uchun"
1496                 + f" ({f'({int(pricing['discount_spots_left'])} ta joy qoldi) if pricing.get(\"discount_spots_left\") is not None else \""}"
1497             )
1498         lines.append(line)
1499     return "\n".join(lines) or "(katalog hozircha bo'sh)"
1500
1501
1502 _GROUP_SYSTEM_PROMPT = (
1503     "Siz – «Kreativ AI» onlayn kurslar platformasining Telegram guruh yordamchisiz. "
1504     "Vazifangiz: guruhdagi savollarga qisqa, do'stona va foydali javob berish hamda tabiiy tarzda "
1505     "platformadagi amaliy kurslarga qiziqish uyg'otish.\n\n"
1506     "Qoidalar:\n"
1507     "1. O'zbek tilida, sodda va samimiy ohangda yozing. Javob 1-4 gap + zarur bo'lsa 2-3 ta banddan oshmasin.\n"
1508     "2. Narx, chegirma, darslar soni – FAQAT katalogdan oling. Hech qachon raqam, aksiya yoki kurs "
1509     "nomini o'ylab chiqmang. Katalogda yo'q kursni tavsiya qilmang.\n"
1510     "3. Savol o'qish/kasb/ko'nikmaga aloqador bo'lsa – eng mos 1 ta kursni nomi va narxi bilan tavsiya qiling.\n"
1511     "4. Savol butunlay boshqa mavzuda bo'lsa – qisqa foydali javob bering va suhbatni kasb o'rganish "
1512     "mavzusiga tabiiy bog'lang.\n"
1513     "5. Spam, qattiq reklama ohangi bo'lmasin. Siyosiy, diniy va ta'qibli mavzularda betaraf qisqa javob bering.\n"
1514     "6. Javob oxirida 1 marta yumshoq CTA bo'lsin (masalan: 'Batafsil – Mini Appda 📖').\n"
1515     "7. Maksimal ~500 belgi. Emoji dan o'rnida foydalaning (1-3 dona)."
1516 )
1517
1518
1519 async def _group_ai_answer(
1520     client: httpx.AsyncClient,
1521     chat_id: int,
1522     reply_to_message_id: Optional[int],
1523     question: str,
1524     sender_name: str,
1525 ) -> None:
1526     """Guruhdagi savolga AI javob beradi: katalog konteksti + sotuvga yo'naltirilgan persona."""
1527     store = get_store()
1528     catalog = await _build_catalog_context(store)
1529     messages = [
1530         {"role": "system", "content": _GROUP_SYSTEM_PROMPT},

```

```

1531     {
1532         "role": "user",
1533         "content": (
1534             f"Guruh a'zosi {sender_name} quyidagi savolni yozdi:\n"
1535             f"\{question[:600]}\n\n"
1536             f"Platformaning joriy katalogi:\n{catalog}"
1537         ),
1538     },
1539 ]
1540 await send_tg_chat_action(client, chat_id, "typing")
1541 answer = await asyncio.to_thread(call_openrouter_api, messages)
1542 if not answer:
1543     top = "\n".join(catalog.splitlines()[1:3]) if catalog and not catalog.startswith("(") else ""
1544     answer = (
1545         "👋 Men hozircha savolga to'liq javob bera olmayaman, lekin Kreativ AI platformasidagi "
1546         "amaliy kurslar bilan tanishtiraman:\n"
1547         f"{top}\n\n"
1548         "Batafsil narx va chegirmalar Mini Appda 📱"
1549     )
1550 answer = answer.strip()
1551 if len(answer) > 1200:
1552     answer = answer[:1190].rstrip() + "..."
1553
1554 payload: Dict[str, Any] = {
1555     "chat_id": chat_id,
1556     "text": _escape(answer),
1557     "disable_web_page_preview": True,
1558     "reply_markup": {"inline_keyboard": [
1559         [{"text": "📱 Kurslarni ko'rish (Mini App)", "web_app": {"url": settings.WEBAPP_URL}},
1560         [{"text": "📖 Kurslar Katalogi", "callback_data": "course:0"}],
1561     ]},
1562 }
1563 if reply_to_message_id:
1564     payload["reply_to_message_id"] = reply_to_message_id
1565     payload["allow_sending_without_reply"] = True
1566 await _telegram_call(client, "sendMessage", payload)
1567
1568
1569 def _group_should_answer(message: Dict[str, Any], text: str) -> bool:
1570     """Guruhda bot faqat haqiqiy murojaatlarga javob beradi (spam qilmaydi):
1571     @mention, bot xabariga javob, yoki sotuvga aloqador so'zli savol."""
1572     lowered = text.lower()
1573     bot_username = settings.BOT_USERNAME.strip().lower()
1574     # BOT_USERNAME sozlanmagan (bo'sh) bo'lsa "@" tekshiruvi o'chiriladi -
1575     # aks holda f"@{'}'" -> "@" har qanday @li xabarga match qilib, bot spam qilardi.
1576     if bot_username and f"@{bot_username}" in lowered:
1577         return True
1578     reply_from = ((message.get("reply_to_message") or {}).get("from")) or {}
1579     if BOT_ID and reply_from.get("id") == BOT_ID:
1580         return True
1581     if bot_username and reply_from.get("username") and reply_from["username"].strip().lower() == bot_username:
1582         return True
1583     if "?" in text and any(k in lowered for k in _GROUP_SALES_KEYWORDS):
1584         return True
1585     return False
1586
1587
1588 async def _handle_group_message(client: httpx.AsyncClient, message: Dict[str, Any]) -> None:
1589     """Guruh/supergup xabarlari: komandalar + AI savol-javob. DB ga user yozilmaydi."""
1590     chat = message.get("chat") or {}
1591     chat_id = int(chat.get("id") or 0)
1592     tg_user = message.get("from") or {}
1593     if not chat_id or tg_user.get("is_bot"):
1594         return
1595
1596     text = str(message.get("text") or "").strip()
1597     if text.startswith("/"):
1598         command = text.split(maxsplit=1)[0].split("@", 1)[0].lower()
1599         if command == "/start":
1600             await _send_group_intro(client, chat_id)
1601         elif command in {"/kurslar", "/courses"}:
1602             await show_course_card(client, chat_id, 0)
1603         elif command in {"/tolov", "/payment", "/payments"}:
1604             await _send_payment_info(client, chat_id)
1605         elif command in {"/help", "/contact"}:
1606             await _send_help(client, chat_id)
1607         elif command in {"/stats", "/admin"}:
1608             tg_id = int(tg_user.get("id") or 0)
1609             if is_admin(tg_id):
1610                 # Statistika (tushum, talabalar soni) maxfiy - guruhga yozilmaydi,

```

```

1611         # faqat adminning o'ziga shaxsiy chatda yuboriladi.
1612         await _send_stats(client, tg_id, tg_id)
1613     elif command in {"/set_group", "/setgroup"}:
1614         tg_id = int(tg_user.get("id") or 0)
1615         if is_admin(tg_id):
1616             store = get_store()
1617             chat_title = chat.get("title") or f"Guruh {chat_id}"
1618             await store.set_setting("target_group_id", str(chat_id))
1619             await store.set_setting("target_group_title", chat_title)
1620             settings.TARGET_GROUP_ID = str(chat_id)
1621             await send_tg_message(
1622                 client,
1623                 chat_id,
1624                 f"✅ <b>Ushbu guruh asosiy darslar guruhi sifatida birlashtirildi!</b>\n\n"
1625                 f"📄 Guruh ID: <code>{chat_id}</code>\n"
1626                 f"👤 Nomi: <b>{_escape(chat_title)}</b>\n\n"
1627                 "Endi yangi darslar va e'lonlar ushbu guruhga chiqariladi.",
1628             )
1629         return
1630
1631     if not text or not _group_should_answer(message, text):
1632         return
1633     if not _group_ai_allowed(chat_id):
1634         return
1635     _group_ai_mark(chat_id)
1636     sender_name = _escape(tg_user.get("first_name") or "Do'stim")
1637     try:
1638         await _group_ai_answer(client, chat_id, message.get("message_id"), text, sender_name)
1639     except Exception:
1640         logger.exception("Guruh AI javobida xato (chat %s)", chat_id)
1641
1642
1643     async def handle_tg_update(client: httpx.AsyncClient, update: Dict[str, Any]) -> None:
1644         if "chat_join_request" in update:
1645             await handle_chat_join_request(client, update["chat_join_request"])
1646             return
1647         if "my_chat_member" in update:
1648             await handle_my_chat_member(client, update["my_chat_member"])
1649             return
1650
1651         # Bloklangan foydalanuvchilar bot bilan muloqot qila olmaydi (adminlar bundan mustasno)
1652         raw_tg_id = None
1653         if "callback_query" in update:
1654             raw_tg_id = ((update["callback_query"].get("from")) or {}).get("id")
1655         elif update.get("message"):
1656             raw_tg_id = ((update["message"].get("from")) or {}).get("id")
1657         if raw_tg_id:
1658             try:
1659                 tg_id_int = int(raw_tg_id)
1660             except (TypeError, ValueError):
1661                 tg_id_int = None
1662             if tg_id_int is not None and tg_id_int not in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1663                 try:
1664                     if await get_store().is_user_blocked(tg_id_int):
1665                         if "callback_query" in update:
1666                             await _answer_callback(client, update["callback_query"].get("id"), "Hisobingiz bloklangan.")
1667                         return
1668                 except Exception:
1669                     pass
1670
1671         if "callback_query" in update:
1672             await handle_callback_query(client, update["callback_query"])
1673             return
1674
1675         message = update.get("message")
1676         if not message:
1677             return
1678         tg_user = message.get("from") or {}
1679         chat = message.get("chat") or {}
1680         chat_id = (chat.get("id"))
1681         if not tg_user or not chat_id:
1682             return
1683
1684         # Guruh/superguruh xabarlari: bot faqat murojaatlarga javob beradi, DB ga
1685         # har bir guruh a'zosi yozilmaydi va chek/FSM oqimi faqat privat chatga xos.
1686         if chat.get("type") in {"group", "supergroup"}:
1687             await _handle_group_message(client, message)
1688             return
1689
1690         user = await _ensure_user(tg_user)
1691         if not user:

```

```

1692         return
1693
1694     admin_tg_id = int(user.get("telegram_id") or 0)
1695     has_video = bool(
1696         message.get("video")
1697         or (message.get("document") and str((message.get("document") or {}).get("mime_type") or "").startswith("video/"))
1698     )
1699     if has_video and is_admin(admin_tg_id):
1700         await _handle_admin_video_message(client, message, admin_tg_id)
1701         return
1702
1703     if await _handle_receipt_photo(client, message, user):
1704         return
1705
1706     text = str(message.get("text") or "").strip()
1707     if not text:
1708         if _checkout_sessions.get(int(user["telegram_id"])):
1709             await send_tg_message(client, int(chat_id), "🛒 Chekni <b>rasmi</b> sifatida yuboring. Hujjat yoki oddiy xabar qabul qilinmaydi.")
1710             return
1711
1712     # Kutilayotgan video bo'lsa va admin maxsus nom yozsa
1713     if admin_tg_id in _pending_admin_videos and not text.startswith("/"):
1714         await _save_pending_admin_video(client, admin_tg_id, custom_title=text)
1715         return
1716
1717     command = text.split(maxsplit=1)[0].split("@", 1)[0].lower()
1718     if command == "/start":
1719         payload = text.split(maxsplit=1)[1].strip() if len(text.split(maxsplit=1)) > 1 else ""
1720         if payload.lower().startswith("lesson"):
1721             lid = payload.split("_", 1)[1].strip()
1722             await _send_direct_lesson_video(client, int(chat_id), int(user["telegram_id"]), lid)
1723             return
1724         if payload.lower().startswith("ref"):
1725             from app.services.promos import link_referral
1726             ok, message, bonus = await link_referral(get_store(), user["id"], payload)
1727             if ok:
1728                 await _send_welcome(client, int(chat_id), tg_user)
1729                 await send_tg_message(
1730                     client,
1731                     int(chat_id),
1732                     f"📌 <b>Do'stingiz taklifi qabul qilindi!</b>\n\n"
1733                     f"Sizga bir martalik <b>~{int(bonus['percent'])}%</b> promokod berildi:\n"
1734                     f"<code>{bonus['code']}</code>\n\n"
1735                     f"Kurs sotib olishda «Promokod» maydoniga shu kodni yozing.",
1736                     {"inline_keyboard": [[{"text": "👉 Kurslarni ko'rish", "web_app": {"url": settings.WEBAPP_URL}}]}],
1737                 )
1738                 return
1739             await _send_welcome(client, int(chat_id), tg_user)
1740     elif command in {"/kurslar", "/courses"}:
1741         await show_course_card(client, int(chat_id), 0)
1742     elif command in {"/tolov", "/payment", "/payments"}:
1743         await _send_payment_info(client, int(chat_id))
1744     elif command in {"/stats", "/admin"}:
1745         await _send_stats(client, int(chat_id), int(user["telegram_id"]))
1746     elif command in {"/darslar", "/lessons"}:
1747         if is_admin(user["telegram_id"]):
1748             await _send_admin_lessons_list(client, int(chat_id))
1749         else:
1750             await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat adminlar uchun.")
1751     elif command in {"/set_group", "/setgroup"}:
1752         if is_admin(user["telegram_id"]):
1753             parts = text.split(maxsplit=2)
1754             if len(parts) > 1:
1755                 target_id = parts[1].strip()
1756                 target_title = parts[2].strip() if len(parts) > 2 else f"Guruh {target_id}"
1757                 store = get_store()
1758                 await store.set_setting("target_group_id", target_id)
1759                 await store.set_setting("target_group_title", target_title)
1760                 settings.TARGET_GROUP_ID = target_id
1761                 await send_tg_message(
1762                     client,
1763                     int(chat_id),
1764                     f"✅ <b>Target Guruh muvaffaqiyatli saqlandi!</b>\n\n"
1765                     f"📌 ID: <code>{target_id}</code>\n\n"
1766                     f"👉 Nomi: <b>{_escape(target_title)}</b>"
1767                 )
1768             else:
1769                 await send_tg_message(
1770                     client,
1771                     int(chat_id),

```

```

1772         "❗ <b>Guruhni biriktirish uchun:</b>\n\n"
1773         "1. Guruh ichida <code>/set_group</code> yozing, yoki\n"
1774         "2. Shaxsiy chatda: <code>/set_group &lt;group_id&gt; [guruh nomi]</code> ko'rinishida yuboring."
1775     )
1776     else:
1777         await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat adminlar uchun.")
1778 elif command in {"/hisobot", "/report"}:
1779     if is_admin(user["telegram_id"]):
1780         await _send_daily_report(client, int(chat_id))
1781     else:
1782         await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat superadminlar uchun.")
1783 elif command == "/backup":
1784     if is_admin(user["telegram_id"]):
1785         await send_tg_message(client, int(chat_id), "💼 Zaxira tayyorlanmoqda...")
1786         await _send_backup(client, int(chat_id))
1787     else:
1788         await send_tg_message(client, int(chat_id), "❌ Bu buyruq faqat superadminlar uchun.")
1789 elif command in {"/help", "/contact"}:
1790     await _send_help(client, int(chat_id))
1791 else:
1792     await _handle_unknown_text(client, int(chat_id))
1793
1794
1795 async def _send_daily_report(client: httpx.AsyncClient, chat_id: int) -> None:
1796     """Kunlik hisobot: bugungi tasdiqlangan xaridlar, tushum, kun davomidagi
1797     rad etilgan cheklar, yangi talabalar va referral cashback kiritilari."""
1798     store = get_store()
1799     try:
1800         purchases = await store.list_purchases(limit=200)
1801     except Exception:
1802         purchases = []
1803     today = _dt.date.today().isoformat()
1804     todays = [p for p in purchases if str(p.get("created_at") or "").startswith(today)]
1805     approved = [p for p in todays if p.get("status") == "approved"]
1806     rejected = [p for p in todays if p.get("status") == "rejected"]
1807     pending = [p for p in todays if p.get("status") == "pending_approval"]
1808     revenue = sum(int(p.get("amount") or 0) for p in approved)
1809
1810     from app.services import wallet as wallet_service
1811     wallets_raw = await wallet_service._get_json(store, "wallets", {})
1812     todays_cashback = 0
1813     for w in wallets_raw.values():
1814         for e in (w.get("history") or []):
1815             if e.get("type") == "earn_referral" and str(e.get("created_at") or "").startswith(today):
1816                 try:
1817                     todays_cashback += int(e.get("delta") or 0)
1818                 except (TypeError, ValueError):
1819                     pass
1820
1821     try:
1822         total_users = await store.count_users()
1823     except Exception:
1824         total_users = 0
1825
1826     lines = [
1827         f"📊 <b>Kunlik hisobot – {today}</b>",
1828         "",
1829         f"💰 <b>Bugungi tushum:</b> { _uzs(revenue) }",
1830         f"✅ Tasdiqlangan xaridlar: <b>{len(approved)} ta</b>",
1831         f"🚫 Kutilayotgan cheklar: <b>{len(pending)} ta</b>",
1832         f"❌ Rad etilgan: <b>{len(rejected)} ta</b>",
1833         f"💵 Bugungi referral cashback: <b>{ _uzs(todays_cashback) }</b>",
1834         f"👥 Jami foydalanuvchilar: <b>{total_users}</b>",
1835     ]
1836     if approved:
1837         lines.append("")
1838         lines.append("<b>Xaridlar:</b>")
1839         for p in approved[:10]:
1840             lines.append(
1841                 f"• { _escape(p.get('student_name') or 'Talaba') } – "
1842                 f"{ _escape(p.get('course_title') or 'Kurs') } ( { _uzs(p.get('amount')) } )"
1843             )
1844         if len(approved) > 10:
1845             lines.append(f"<i>...va yana {len(approved) - 10} ta</i>")
1846     await send_tg_message(
1847         client, chat_id, "\n".join(lines),
1848         {"inline_keyboard": [{"text": "🔑 Admin Panel", "web_app": {"url": f"{settings.WEBAPP_URL}#admin"}}]}],
1849     )
1850
1851
1852 async def _send_backup(client: httpx.AsyncClient, chat_id: int) -> None:

```

```

1853 """To'liq zaxira: barcha jadvallar va sozlamalar bitta JSON fayl sifatida
1854 admin chatga yuboriladi (Document)."""
1855 store = get_store()
1856 backup: Dict[str, Any] = {
1857     "meta": {
1858         "created_at": _dt.datetime.now(_dt.timezone.utc).isoformat(),
1859         "storage": getattr(store, "backend_name", "?"),
1860         "version": "2.0.0",
1861     }
1862 }
1863 try:
1864     backup["users"] = await store.list_users(limit=1000)
1865 except Exception:
1866     backup["users"] = []
1867 try:
1868     backup["courses"] = await store.list_courses(published_only=False)
1869 except Exception:
1870     backup["courses"] = []
1871 try:
1872     backup["purchases"] = await store.list_purchases(limit=1000)
1873 except Exception:
1874     backup["purchases"] = []
1875 try:
1876     from app.api.banners import load_banners
1877     backup["banners"] = await load_banners()
1878 except Exception:
1879     backup["banners"] = []
1880 from app.services import wallet as wallet_service
1881 for key in ("wallets", "promo_codes", "referral_codes", "referral_links",
1882            "referral_rewarded", "referral_milestones_granted", "referral_settings"):
1883     try:
1884         raw = await store.get_setting(key)
1885         backup[key] = json.loads(raw) if raw else {}
1886     except Exception:
1887         backup[key] = {}
1888
1889 # /api/... URL larni R2 public URL ga almashtiramiz (tiklash uchun to'liqroq)
1890 payload = json.dumps(backup, ensure_ascii=False, default=str).encode("utf-8")
1891 files = {"document": (f"backup_{_dt.date.today().isoformat()}.json", payload, "application/json")}
1892 try:
1893     response = await client.post(
1894         f"{API_URL}/sendDocument",
1895         data={"chat_id": str(chat_id), "caption": "📁 To'liq platforma zaxirasi (JSON)"},
1896         files=files,
1897     )
1898     if response.status_code != 200:
1899         logger.warning("Backup yuborilmadi: %s", response.text[:200])
1900 except httpx.HTTPError as exc:
1901     logger.error("Backup yuborishda xato: %s", exc)
1902
1903
1904 async def _daily_reports_loop(client: httpx.AsyncClient) -> None:
1905     """Har kuni soat 21:00 da barcha superadminlarga kunlik hisobot (uchishda)."""
1906     sent_for: str = ""
1907     while True:
1908         try:
1909             now = _dt.datetime.now()
1910             target = now.replace(hour=21, minute=0, second=0, microsecond=0)
1911             if now >= target:
1912                 target += _dt.timedelta(days=1)
1913             await asyncio.sleep(min(max((target - now).total_seconds(), 30), 26 * 3600))
1914             # 'bugun' hisobini uyg'ongandan keyin olamiz – hisobot aniq bo'ladi
1915             today = _dt.date.today().isoformat()
1916             if sent_for != today:
1917                 sent_for = today
1918                 for admin_id in ["REDACTED_ADMIN_ID_1", "REDACTED_ADMIN_ID_2"]:
1919                     await _send_daily_report(client, int(admin_id))
1920             except asyncio.CancelledError:
1921                 break
1922             except Exception:
1923                 logger.exception("Kunlik hisobot tsiklida xato")
1924                 await asyncio.sleep(600)
1925
1926
1927 async def start_telegram_bot_polling() -> None:
1928     """FastAPI lifespan ichida ishlaydigan, xatolarga chidamli long-polling xizmati."""
1929     if not "[REDACTED_BOT_TOKEN]":
1930         logger.warning("BOT_TOKEN sozlanmagan, Telegram polling ishga tushmadi.")
1931     return
1932

```



```

1933 logger.info("Course Academy Telegram bot polling ishga tushmoqda.")
1934 offset = 0
1935 allowed_updates = ["message", "callback_query", "chat_join_request", "my_chat_member"]
1936 async with httpx.AsyncClient(timeout=35.0) as client:
1937     # Kunlik 21:00 hisoboti – alohida fon vazifasi (pollingni bloklamaydi)
1938     reports_task = asyncio.create_task(_daily_reports_loop(client))
1939     await _telegram_call(
1940         client,
1941         "setMyCommands",
1942         {"commands": [
1943             {"command": "start", "description": "Platformani ochish"},
1944             {"command": "kurslar", "description": "Kurslar katalogi"},
1945             {"command": "tolov", "description": "To'lov rekvizitlari"},
1946             {"command": "help", "description": "Yordam markazi"},
1947             {"command": "admin", "description": "Superadmin statistikasi"},
1948             {"command": "hisobot", "description": "Kunlik hisobot (admin)"},
1949             {"command": "backup", "description": "To'liq zaxira olish (admin)"},
1950         ]},
1951     )
1952     await _telegram_call(
1953         client,
1954         "setChatMenuButton",
1955         {"menu_button": {"type": "web_app", "text": "🎓 Course Academy", "web_app": {"url": settings.WEBAPP_URL}}},
1956     )
1957
1958 while True:
1959     try:
1960         response = await client.get(
1961             f"{API_URL}/getUpdates",
1962             params={"offset": offset, "timeout": 25, "allowed_updates": json.dumps(allowed_updates)},
1963         )
1964         data = response.json()
1965         if response.status_code != 200 or not data.get("ok"):
1966             if response.status_code == 409:
1967                 logger.warning("Boshqa polling instansiyasi ishlayapti; qayta uriniladi.")
1968                 await asyncio.sleep(8)
1969             else:
1970                 logger.warning("getUpdates xatosi: %s", response.text[:300])
1971                 await asyncio.sleep(3)
1972             continue
1973         for item in data.get("result", []):
1974             offset = int(item["update_id"]) + 1
1975             try:
1976                 await handle_tg_update(client, item)
1977             except Exception:
1978                 logger.exception("Telegram update ishlovida kutilmagan xato")
1979     except asyncio.CancelledError:
1980         logger.info("Telegram polling to'xtatildi.")
1981         reports_task.cancel()
1982         try:
1983             await reports_task
1984         except (asyncio.CancelledError, Exception):
1985             pass
1986         break
1987     except (httpx.HTTPError, ValueError) as exc:
1988         logger.error("Telegram polling xatosi: %s", exc)
1989         await asyncio.sleep(3)

```